City Brain Convergence: System Architecture

1 Overall Architecture and Inter-Layer Communication

The system is composed of three independently developed layers, the **Twin Layer**, the **City Brain Layer**, and the **Guardian Layer**. Interoperability is achieved exclusively through **Model Context Protocol (MCP)** servers [1]: each cross-layer interface is a JSON-RPC 2.0 [2] endpoint exposing a typed `tools/list` schema and invoked through `tools/call`, so that a calling agent discovers available operations and their input/output contracts at runtime rather than through hard-coded bindings.[1] This is what allows three teams with no shared codebase to interoperate: the contract is the tool schema, not the implementation behind it.

1.1 Protocol stack and transport security

Three categories of MCP surface exist. The **External MCP** (Twin-owned) is the boundary reachable by the TSYP evaluation committee, used to invoke Simulator and Actuator tools. **Control MCP** servers (one per layer) expose privileged, state-mutating operations (spawn/despawn a subject, freeze an agent, patch context) to the other two layers. **Logs MCP** is a read-only, resource-oriented interface (MCP `resources/read`) used for on-demand drill-down into an already-streamed log line. Continuous telemetry does not go through MCP request/response at all: it is pushed as a persistent **log stream** (newline-delimited JSON, NDJSON) over a TLS 1.3 [3] channel, each connection authenticated by mutual TLS (mTLS) client certificates so that a receiving layer can cryptographically verify the identity of the layer it is receiving from, not merely encrypt the payload.[2] Every NDJSON record carries a monotonic sequence number and an HMAC [4] computed over the record; this lets the Guardian Layer detect dropped, reordered, duplicated, or altered lines without needing to reconstruct application-level meaning, which is the mechanism referenced in Section 4 as log cross-validation.[3]

1.2 Trust and control loop

City Brain's actuator commands and Guardian's containment commands both terminate in Twin's Control MCP, but they are not symmetric: City Brain proposes, Guardian verifies. The two Control MCP channels (City-Brain-to-Twin and Guardian-to-City-Brain) implement a **verify-before-act** pattern, structurally equivalent to a policy-enforcement point in a zero-trust architecture [5], where Guardian's Master Agent is the policy decision point and City Brain's Master Agent is the enforcement point that must honor a freeze, context patch, or discrepancy flag before its own actuator call is allowed to reach Twin.[4]

[1] See [1], "Tools" specification, and [2], §4 (Request object), for the exact schema of a `tools/call` message.
[2] Client-certificate mutual authentication within the handshake is specified in [3], §4.4.2 (Certificate Verify).
[3] The HMAC construction and its security proof are given in [4], §2–3.
[4] The policy decision point / policy enforcement point split follows the logical architecture in [5], §3.2, Fig. 2.

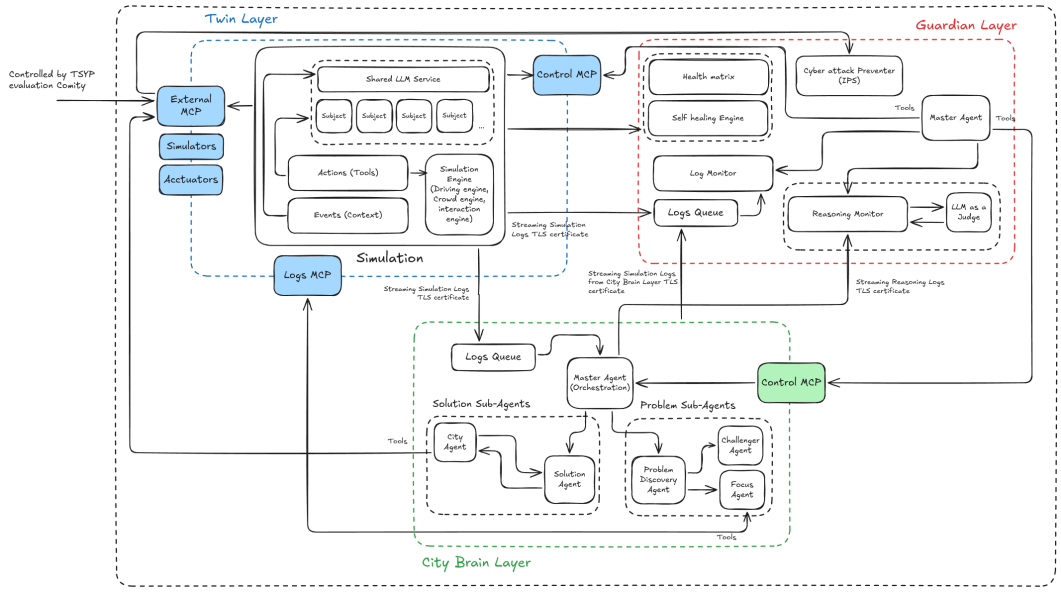


Figure 1: Full system architecture, cross-layer MCP interfaces, and TLS-secured log/reasoning streams.

2 Twin Layer

2.1 Trait-conditioned agent policy

Each Subject (vehicle, pedestrian, emergency unit) is instantiated with a randomized **trait vector** $\mathbf{t} \in [0,1]^d$, a fixed-dimensional behavioral encoding analogous to a personality-trait embedding (in the spirit of an OCEAN-style trait basis, generalized to arbitrary custom axes such as urgency, risk-aversion, or compliance). At each simulation tick, a subject's next action is a function $\pi(\mathbf{t}, \mathbf{c}, \mathcal{A})$ of its trait vector, the current context vector $\mathbf{c}$ (time, weather, active event flags), and its bounded discrete action space $\mathcal{A}$ (the tool set exposed to that subject class). This is formally a **multi-class policy over a discrete action space**, the same structure as a behavior-cloned control policy in imitation learning [9]. How π itself is produced and served is detailed in Section 3.

2.2 Decision/physics separation and domain engines

The policy layer only emits an **intent** (a discrete action token, e.g. `GOTO(park)`); it never integrates motion itself. Intents are dispatched to domain-specific physics engines: vehicle motion is delegated to **SUMO (Simulation of Urban MObility)** through its **TraCI (Traffic Control Interface)** protocol [12], a TCP client/server API (default port 8813) that steps the simulation and exchanges per-vehicle state; SUMO's internal car-following model (Krauss model) [13] and lane-changing model govern low-level kinematics, so the Twin's policy layer never has to reason about collision avoidance or headway directly.[5] Pedestrian and crowd dynamics are handled by a separate engine, conceptually a **social-force model** (Helbing-Molnár attractive/repulsive force fields) [14] for local collision avoidance and flocking.[6] An interaction engine resolves cross-domain effects (a fire spreading, a jam backing into a side street). Engine state is streamed to a Visual Layer over a publish/subscribe channel for real-time rendering, decoupled from the authoritative simulation state.

2.3 External surfaces

The External MCP exposes **Simulators** (idempotent tool calls that mutate the context vector or inject event flags, e.g. `set_event(flood=true)`) and **Actuators** (tool calls that instantiate or redirect a privileged subject, e.g. `dispatch(ambulance, x, y)`). *Example*: a collision is detected

[5] SUMO/TraCI's architecture and coupling API are described in [12], pp. 2575–2582; the safe-speed car-following rule it implements by default is derived in [13], ch. 2–3.

[6] The desired/social/obstacle force decomposition is given in [14], pp. 4282–4286.

when two vehicle subjects report near-zero velocity at overlapping TraCI coordinates for more than a fixed dwell time; an `Actuators.dispatch` call spawns an ambulance subject whose destination is set to the crash coordinates, after which it is governed by the same TraCI-driven driving model as any other vehicle.

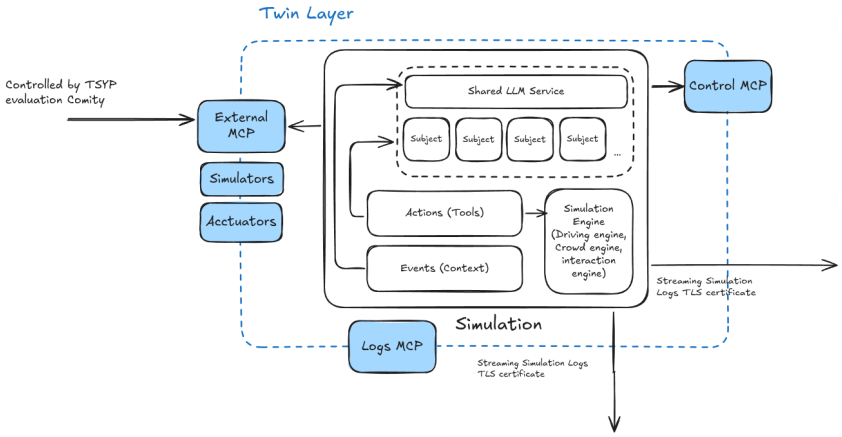


Figure 2: Twin Layer, internal subject/decision pipeline and external surfaces.

3 Teacher LLM and Student Model

This section details how the policy $\pi(\mathbf{t}, \mathbf{c}, \mathcal{A})$ referenced in Section 2.1 is actually produced and served, via **offline knowledge distillation** [8] from a large teacher model into a lightweight student, since it is the component that makes thousands of concurrent, individuated agent decisions computationally tractable.[7]

3.1 Dataset generation via the teacher LLM

An LLM **teacher model** is used purely offline, as a labeling oracle, never in the per-tick decision path in steady state. Training pairs $(\mathbf{t}, \mathbf{c})$ are drawn using a coverage-oriented sampling design, either stratified sampling or Latin hypercube sampling across the trait space $[0, 1]^d$, crossed with every supported event/context combination (including compound events such as a heatwave co-occurring with a power outage), so that the trait/event corners the evaluation committee is most likely to probe adversarially are represented in training, not just the typical case. For each sampled pair, the teacher is prompted with $(\mathbf{t}, \mathbf{c}, \mathcal{A})$, the trait vector rendered as a natural-language persona, the context as a structured scenario description, and the action space as an enumerated tool list, and asked to choose an action. Rather than a single greedy decode, each pair is queried multiple times under temperature sampling (or, where the API exposes it, top-k log-probabilities over the action tokens directly), yielding an empirical distribution over $\mathcal{A}$ per pair instead of one hard label. Generation is parallelized via batched/asynchronous API calls, since this stage is fully offline and embarrassingly parallel across samples. On the order of 10^3 to 10^4 labeled pairs are generated per scenario family.

3.2 Label and feature design

Each training example is the tuple $(\mathbf{t}, \mathbf{c}, \mathcal{A}, \mathbf{y})$, where $\mathbf{y}$ is the teacher's empirical action distribution (a soft label) rather than a one-hot vector, which preserves the teacher's uncertainty on ambiguous cases instead of forcing an arbitrary tie-break. $\mathcal{A}$ is encoded per subject class as a fixed-width multi-hot mask so the same model architecture can be reused across classes with different available tool sets, with unavailable actions masked out of the loss. Continuous trait and context features are min-max normalized to $[0, 1]$; categorical context flags (event type) are one-hot encoded.

[7] The soft-target distillation objective this pipeline follows (matching a student's output distribution to a teacher's, rather than to hard labels) is introduced in [8], §2.

3.3 Student model training

The **student model** is a gradient-boosted decision tree ensemble (**XGBoost**) [6], trained with a multi-class, multi-label-masked softmax objective (`multi:softprob`), which fits an additive ensemble of regression trees via second-order (Newton) gradient boosting [7], at each boosting round fitting new trees to the gradient and Hessian of the multi-class log-loss between the current ensemble's predicted distribution and the teacher's soft label $\mathbf{y}$.[8] Model capacity (tree depth, number of boosting rounds, learning rate/shrinkage, L1/L2 leaf-weight regularization) is tuned against a held-out validation split using early stopping on validation log-loss, to avoid overfitting the comparatively small teacher-labeled set. Per-feature gain-based importance is inspected post-training as a sanity check, confirming that trait and event features the design intends to matter (e.g. risk-aversion under a flood flag) actually carry predictive weight, which also gives the team an interpretability artifact for the technical-choice justification required in the challenge report.

3.4 Confidence-gated serving and drift monitoring

At simulation runtime, each subject's $(\mathbf{t}, \mathbf{c})$ is passed through the trained student in a single batched, vectorized `predict_proba` call covering all subjects due for a decision that tick, which is orders of magnitude cheaper than an equivalent number of individual LLM round-trips. Serving follows a **selective prediction / model cascade** [10, 11]: the maximum class probability (or equivalently the Shannon entropy of the full predicted distribution) is thresholded, and only predictions clearing the confidence threshold are accepted directly; sub-threshold cases, novel event combinations or trait regions underrepresented in training, fall back to a live call against the shared LLM service, so quality degrades gracefully rather than silently on distribution shift.[9] Independently of this fallback, a small fixed fraction of accepted student predictions are shadow-checked against the live LLM on every tick (canary sampling); a rising disagreement rate between student and teacher on these canaries is the trigger for regenerating the training set (Section 3.1) and retraining the student, closing the loop between production drift and the offline pipeline.

[8] The second-order (Newton) boosting objective and its regularization terms are derived in [6], §2, pp. 786–787; the general gradient-boosting framework it specializes is due to [7].

[9] The reject-option formalism (accept if confidence exceeds a threshold, else defer) is due to [10]; [11] applies the same cascade principle specifically to cascading cheaper and more expensive LLM calls under a cost budget.

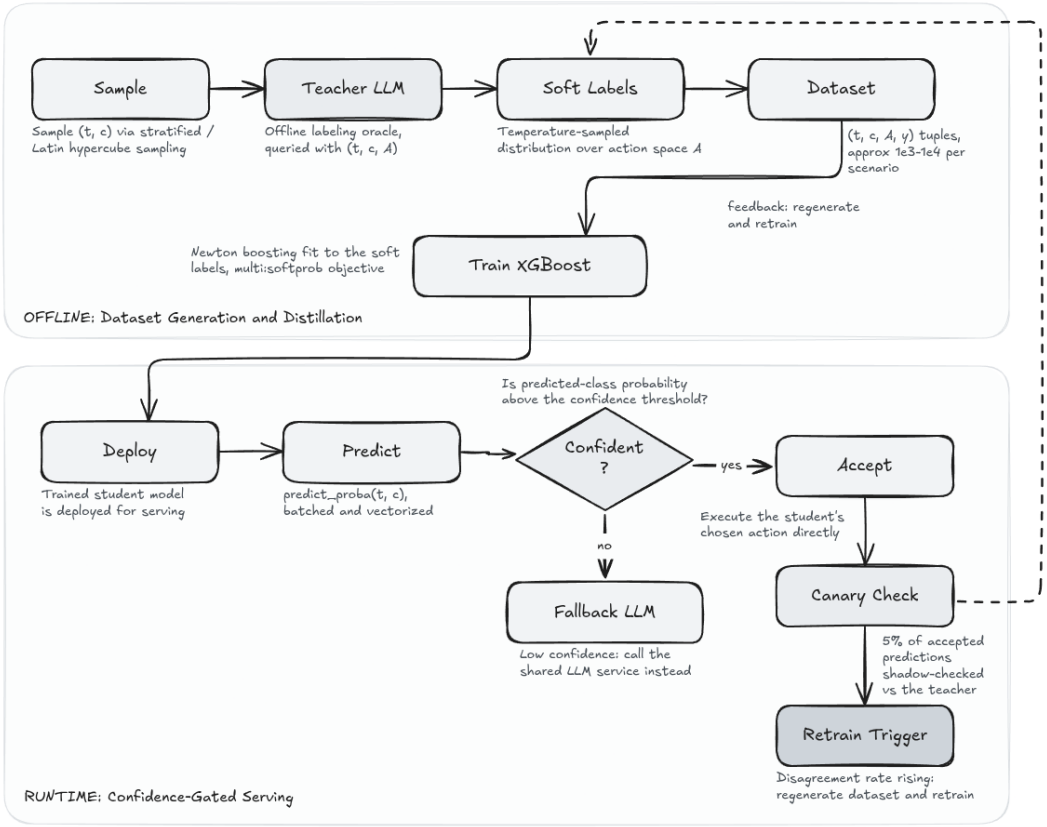


Figure 5: Teacher/student training pipeline, offline distillation (top) and confidence-gated runtime serving with canary-triggered retraining (bottom).

4 City Brain Layer

4.1 Ingestion: micro-batching

Twin's NDJSON log stream is absorbed into a **Logs Queue** implementing sliding-window micro-batching: records are grouped by a size- or time-based trigger (e.g. N lines or a fixed interval Δt, whichever fires first) before being handed to the Master Agent, trading a small amount of latency for a large reduction in orchestration overhead compared to per-line agent invocation. The same raw batch is relayed byte-for-byte to the Guardian Layer over its own TLS/mTLS channel, preserving the sequence numbers and HMACs described in Section 1.2.

4.2 Orchestration

The **Master Agent** implements an orchestrator-worker multi-agent pattern (a directed graph of specialized sub-agents rather than a single monolithic prompt), consistent with a **ReAct-style** (Reason + Act) control loop [15] in which each sub-agent alternates between reasoning steps and tool calls.[10]

4.3 Problem Sub-Agents

The **Problem Discovery Agent** performs hybrid incident detection: lightweight statistical checks (e.g. z-score outlier detection on speed or position deltas across consecutive batches) flag structural anomalies, which are then classified by an LLM few-shot classifier into incident categories (accident, fire, jam, flooding). A positive flag is passed to the **Challenger Agent**, which implements a **generator-critic (self-refine) pattern** [16]: it is prompted adversarially to argue the null hypothesis (this is *not* an incident), reducing false-positive rate before any downstream action

[10] The interleaved thought/action/observation loop is defined in [15], §2–3.

is committed.[11] If the Problem Discovery Agent cannot resolve the disagreement from the current batch alone, the **Focus Agent** performs targeted tool-augmented retrieval, a scoped call to Twin's Logs MCP (equivalent to a narrow, single-purpose RAG lookup) constrained to a specific subject ID or map region, and returns the additional context to the Problem Discovery Agent for a final decision.

4.4 Solution Sub-Agents

A confirmed incident is handed to the **Solution Agent**, which proposes a candidate remediation, and the **City Agent**, a tool-augmented agent holding Twin's External MCP schema as its available action space, which validates feasibility against what is actually executable. The two run a bounded **multi-agent debate / negotiation loop** until they converge on a single action, at which point the City Agent issues the corresponding `tools/call` against Twin's External MCP. *Example*: a confirmed two-vehicle collision at fixed coordinates resolves, after negotiation, to a single `Actuators.dispatch(ambulance, ...)` call.

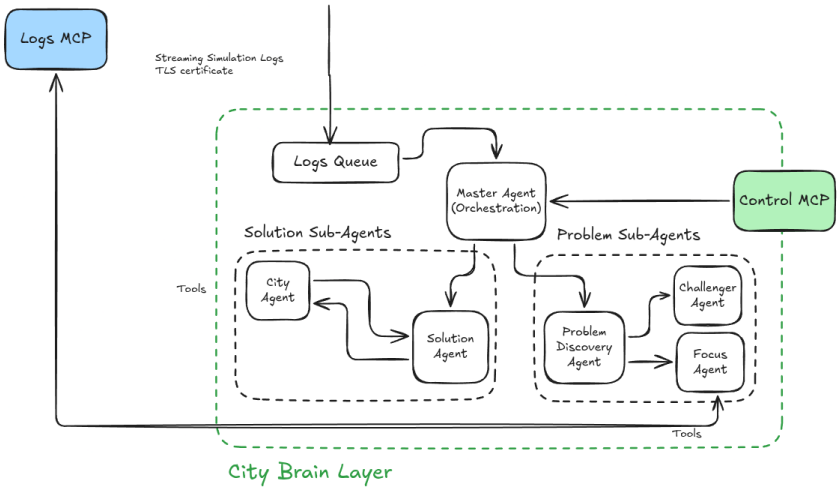


Figure 3: City Brain Layer, Problem and Solution sub-agent workflow.

5 Guardian Layer

5.1 Redundant-path integrity verification

Guardian's Logs Queue ingests two independently transported copies of the same log batch: the Twin-origin stream and the City-Brain-relayed stream. The **Log Monitor** performs a sequence-number and HMAC diff between the two: a mismatch in sequence numbers indicates dropped or reordered lines, and an HMAC mismatch on a matching sequence number indicates content tampering. This is a lightweight, application-level analogue of hash-chain tamper evidence, requiring no shared secret beyond the per-session mTLS-derived key. Any discrepancy is escalated to the Master Agent.

5.2 Reasoning verification: LLM-as-a-judge

City Brain additionally streams its sub-agents' reasoning traces (its ReAct thought/action/observation transcript) to the **Reasoning Monitor**, which applies the **LLM-as-a-judge** technique [17]: an independent LLM scores the transcript against a fixed rubric of security- and safety-relevant criteria (e.g. was the evidence sufficient before acting, was the chosen actuator proportionate, does the reasoning reference data absent from the verified log stream).[12] The resulting verdict, not the

[11] The iterative generate/critique/revise loop this mirrors is proposed in [16], §3.

[12] The rubric-based single-answer grading mode used here follows [17], §4 ("single-answer grading"), rather than the paper's pairwise-comparison mode, since Guardian scores one transcript at a time against fixed criteria rather than comparing two candidate outputs.

raw transcript, is what reaches the Master Agent, which can then exercise City Brain's Control MCP to skip a queue element, patch context, freeze a sub-agent, or attach a discrepancy flag to City Brain's next reasoning cycle.

5.3 Health Matrix and self-healing

The **Health Matrix** tracks standard service-level indicators, latency percentiles ($P_{50}/P_{95}/P_{99}$), request throughput, and queue depth, sourced from Twin's subject-model request path, the simulation engine, and the visual layer, and flags degradation via an exponentially weighted moving average (EWMA) threshold [18] rather than a static cutoff, so that transient spikes are tolerated while sustained regressions trigger action.[13] The **Self-Healing Engine** then applies a **load-shedding / backpressure** strategy analogous to a circuit breaker [19]: a subset of low-priority subjects is temporarily frozen (excluded from the concurrent decision loop), draining the request backlog, and unfrozen once the EWMA returns below threshold.[14] This trades momentary simulation completeness for bounded latency under load.

5.4 Cyber Attack Preventer

The **Cyber Attack Preventer** is an intrusion prevention system (IPS) wired directly to Twin's External MCP, the only surface reachable by outside actors, combining signature-based detection (known malformed-payload and replay patterns, the latter countered by the sequence-number/nonce scheme in Section 1.2) with anomaly-based detection (rate limiting and call-pattern baselining per client certificate). Because tool-call arguments ultimately reach LLM-driven agents downstream, inputs are also schema-validated against each tool's declared JSON schema before execution, closing off malformed or adversarially crafted arguments as a prompt-injection vector into City Brain's or Guardian's own reasoning.

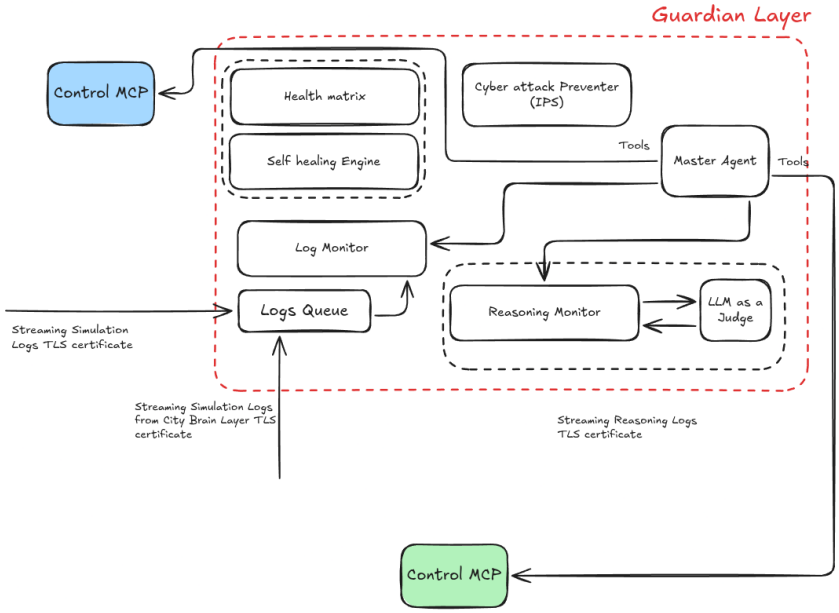


Figure 4: Guardian Layer, log/reasoning verification, health, and cyber defense.

6 Summary

Trust in this architecture is continuously re-derived rather than assumed: Guardian cryptographically cross-validates Twin's ground truth against City Brain's relayed copy, applies LLM-as-a-judge scoring to City Brain's reasoning trace, and holds a direct, schema-typed control channel into City

[13] The geometric moving-average control statistic and its control limits are due to [18].

[14] The circuit-breaker stability pattern this mirrors, tripping under sustained failure/latency and resetting after recovery, is catalogued in [19], ch. 5.

Brain to intervene before an unverified decision reaches a real actuator, while EWMA-triggered load shedding keeps the underlying simulation within latency bounds. Twin's own cost profile is kept tractable at scale through confidence-gated policy distillation from an LLM teacher into a boosted-tree student, with physics delegated to domain-appropriate engines such as SUMO/TraCI. Together these mechanisms satisfy the challenge's core requirement: three independently built systems interoperating over a shared, typed, cryptographically verifiable technical contract, including when the data itself is being falsified.

References

[1] Anthropic. *Model Context Protocol Specification*. 2024. "Tools" and "Resources" primitives, `tools/list`/`tools/call`/`resources/read` methods.

[2] JSON-RPC Working Group. *JSON-RPC 2.0 Specification*. 2010. §4 (Request object), §5 (Response object).

[3] E. Rescorla. *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 8446, IETF, 2018. §4 (Handshake Protocol), §4.4.2 (Certificate Verify, mutual authentication).

[4] M. Bellare, R. Canetti, H. Krawczyk. "Keying Hash Functions for Message Authentication." *CRYPTO 1996*, pp. 1–15. Standardized as RFC 2104.

[5] S. Rose, O. Borchert, S. Mitchell, S. Connelly. *Zero Trust Architecture*. NIST Special Publication 800-207, 2020. §3.2 (Logical Components, policy decision/enforcement points), Fig. 2.

[6] T. Chen, C. Guestrin. "XGBoost: A Scalable Tree Boosting System." *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*, 2016, pp. 785–794.

[7] J. H. Friedman. "Greedy Function Approximation: A Gradient Boosting Machine." *Annals of Statistics*, 29(5), 2001, pp. 1189–1232.

[8] G. Hinton, O. Vinyals, J. Dean. "Distilling the Knowledge in a Neural Network." *arXiv:1503.02531*, 2015, §2 (soft targets).

[9] D. A. Pomerleau. "ALVINN: An Autonomous Land Vehicle in a Neural Network." *Advances in Neural Information Processing Systems (NeurIPS) 1*, 1988, pp. 305–313.

[10] Y. Geifman, R. El-Yaniv. "Selective Classification for Deep Neural Networks." *Advances in Neural Information Processing Systems (NeurIPS) 30*, 2017, pp. 4878–4887.

[11] L. Chen, M. Zaharia, J. Zou. "FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance." *arXiv:2305.05176*, 2023, §3 (LLM cascade).

[12] P. A. Lopez et al. "Microscopic Traffic Simulation using SUMO." *2018 21st IEEE International Conference on Intelligent Transportation Systems (ITSC)*, 2018, pp. 2575–2582.

[13] S. Krauss. *Microscopic Modeling of Traffic Flow: Investigation of Collision Free Vehicle Dynamics*. PhD thesis, University of Cologne, 1998. Ch. 2–3 (safe-speed car-following model).

[14] D. Helbing, P. Molnár. "Social Force Model for Pedestrian Dynamics." *Physical Review E*, 51(5), 1995, pp. 4282–4286.

[15] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao. "ReAct: Synergizing Reasoning and Acting in Language Models." *International Conference on Learning Representations (ICLR)*, 2023.

[16] A. Madaan et al. "Self-Refine: Iterative Refinement with Self-Feedback." *Advances in Neural Information Processing Systems (NeurIPS) 36*, 2023.

[17] L. Zheng et al. "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena." *Advances*

in Neural Information Processing Systems (NeurIPS) 36, Datasets and Benchmarks Track, 2023, §4.

[18] S. W. Roberts. "Control Chart Tests Based on Geometric Moving Averages." *Technometrics*, 1(3), 1959, pp. 239–250.

[19] M. T. Nygard. *Release It!: Design and Deploy Production-Ready Software*. Pragmatic Bookshelf, 2007. Ch. 5 (Circuit Breaker).